Ask Learn

100 XP

# Introduction

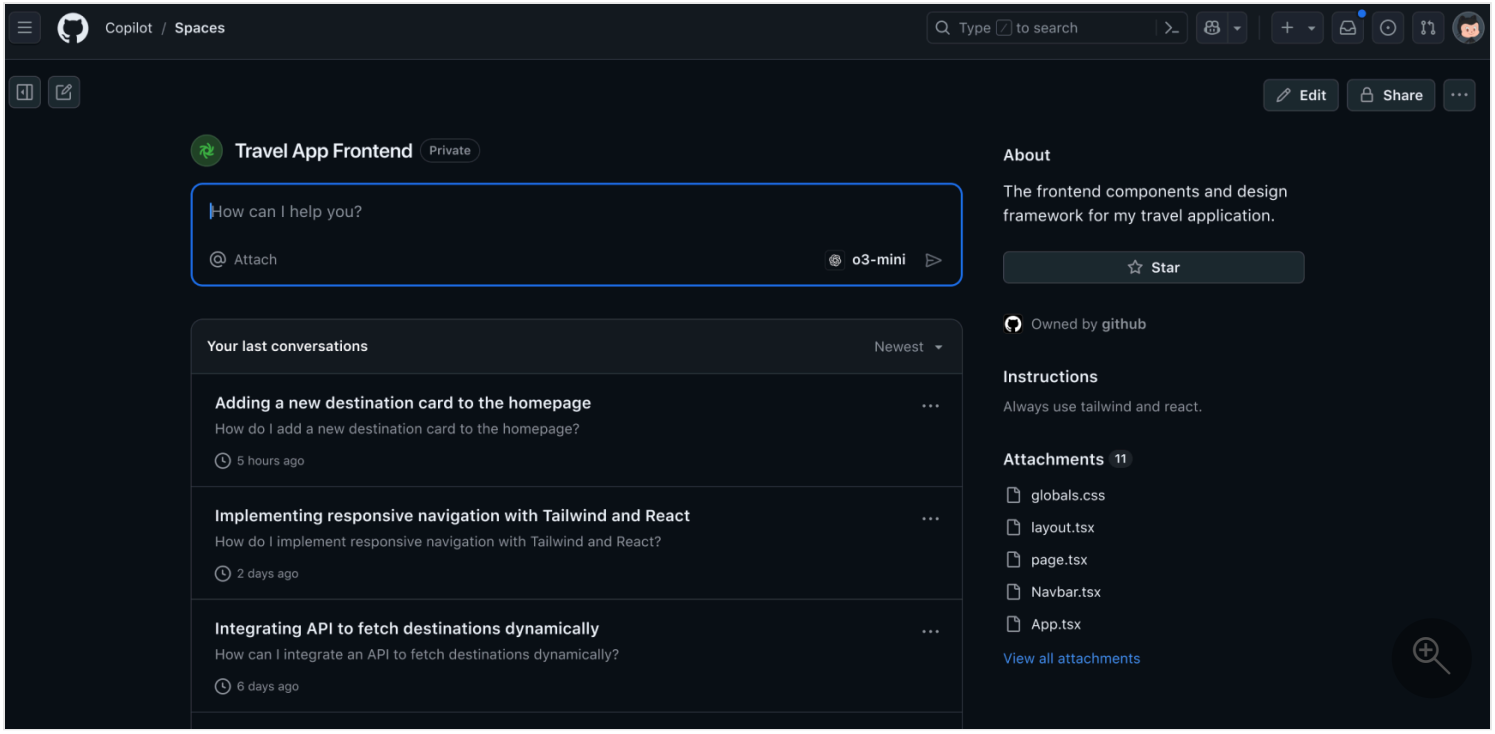
5 minutes

GitHub Copilot Spaces provides a new way to work with AI by anchoring its responses in a carefully curated context. Unlike general Copilot Chat, which surfaces broad suggestions, a Space allows you to focus the model on specific files, issues, pull requests, and tailored instructions. This unit introduces what a Space is, how it works, and why narrowing the context leads to more consistent, reproducible answers. You'll also learn how to set effective context using attachments and free-text instructions, and when it's best to use Spaces over general chat.

In this unit, you'll learn:

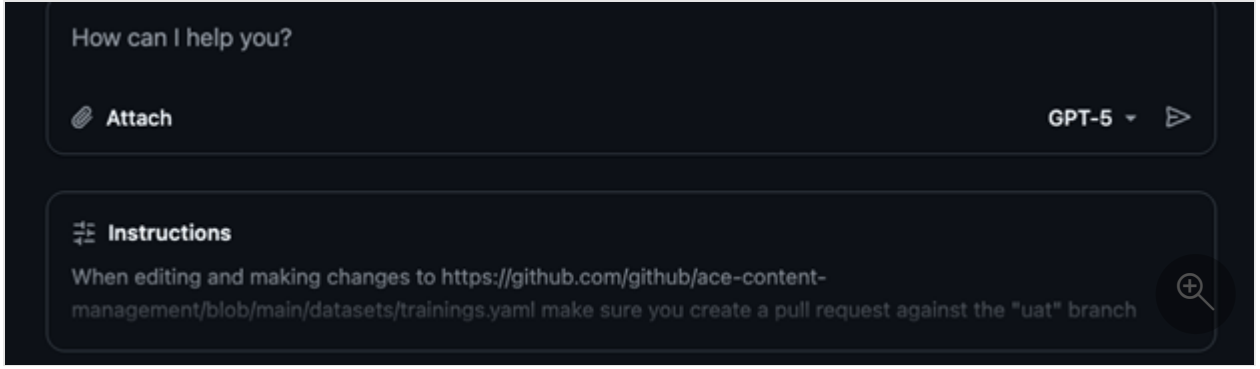
- What GitHub Copilot Spaces are and how they differ from general Copilot Chat
- Why tightly scoped context improves answer quality and consistency
- How to attach files, issues, and instructions to guide the model
- When to create a Space for repeatable, domain-specific tasks

## What is a GitHub Copilot Space?




It's a dedicated Copilot chat grounded in a curated set of context you choose. The Space is itself like a LLM and you can feed it GitHub files, issues, pull requests, and your own free-text instructions to provide context to your specific topic.

## Setting Context for Copilot Spaces



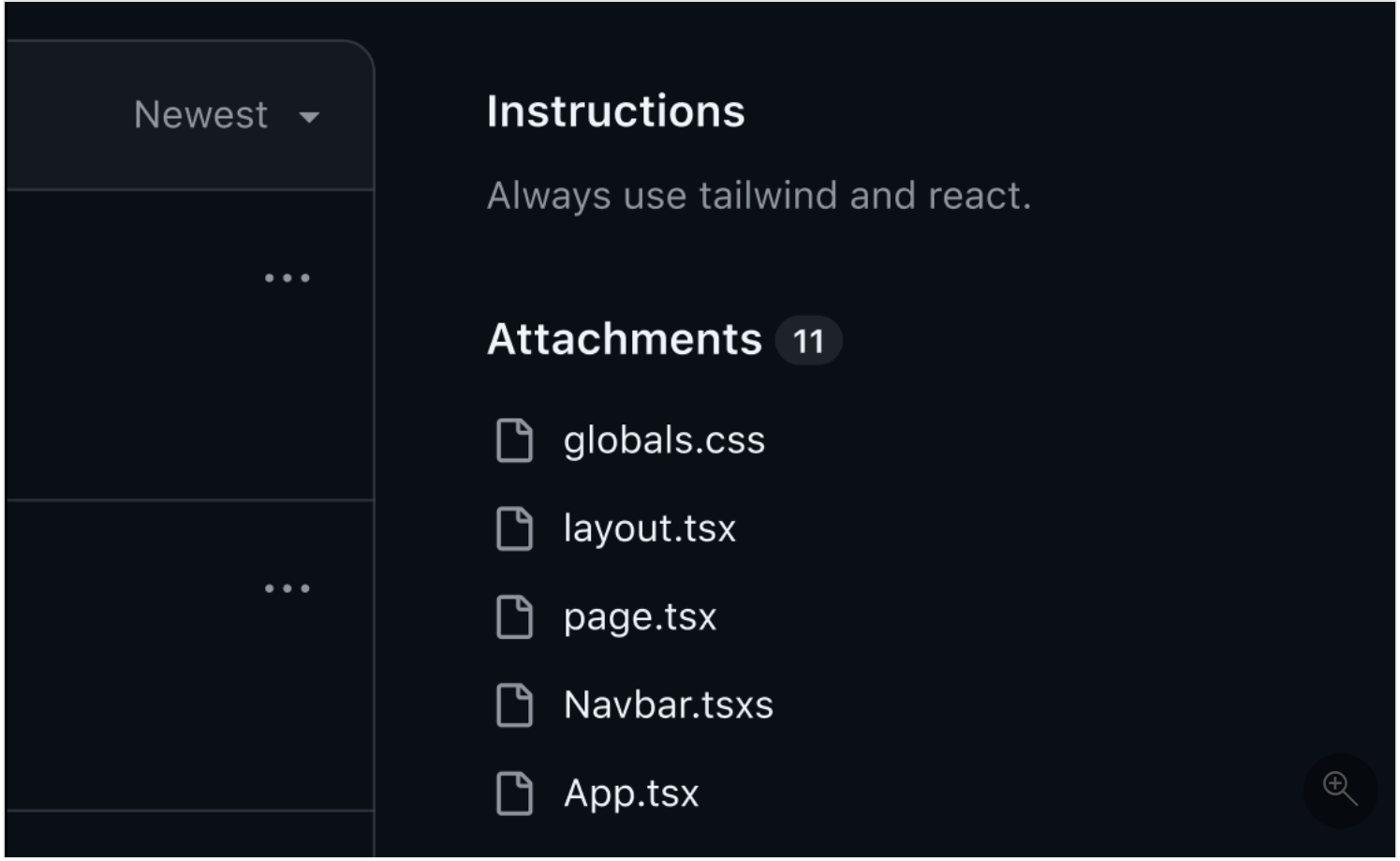

The effectiveness of a Copilot Space depends on the context you provide. You can attach specific files (such as scripts, configuration, or documentation), relevant issues or pull requests, and tailored instructions. By curating this input, you help Copilot focus on the information that matters most for your scenario. The context order matters: leading with the most critical files or instructions helps drive more accurate and relevant responses.

# Setup: Attaching Files (Uploads) and Instructions in Copilot Spaces

**Attaching Files (Uploads):**

- In the Space setup, use the "Attach files" or "Add context" button to select one or more files from your GitHub repository.
- You can attach source code files, markdown docs, configuration files, or other assets as context. These files are referenced from the default branch, so your Space stays up to date as your repo evolves.
- If allowed by your workspace settings, you may also upload files directly (such as images or datasets) from your local machine for non-repo context.

**Adding Instructions:**




- Use the "Instructions" section to provide specific guidance to Copilot. This can include goals ("Summarize the onboarding process"), style preferences ("Write in a formal tone"), or canonical examples ("Sample output should look like...").
- Keep instructions brief, focused, and actionable. If your Space serves a workflow or troubleshooting guide, include step-by-step tasks or sample prompts.
- You can update instructions at any time to refine the focus of your Space.

# The ideal time you want to use and create a GitHub Copilot Spaces

Use a Space when you want consistent, reproducible answers on a tightly scoped topic, like a particular service, a runbook or playbook, or a known dataset. Compared to general or repo-wide chat, Spaces trade breadth for depth: by narrowing the context to what matters most, they tend to produce more predictable, grounded responses, while broad chat can surface wider discovery but may be less precise.

A few practical guidelines improve quality. Model context limits apply, so keep Spaces small and focused. Linked GitHub files reflect the repository's default branch, helping content stay current as code evolves. Be clear and concise with your instructions, and include a few canonical examples to anchor style and expected outputs. Finally, remember that the selection and ordering of context can influence responses, so lead with your most important sources.

---

## Next unit: Creating your first space

Next

100 XP

# Creating your first space

5 minutes

Creating a Space is simple but powerful—once named and configured, it becomes a reusable workspace where Copilot operates within a clearly defined scope. In this unit, you'll walk through how to create your first Space, decide on ownership and visibility, and populate it with meaningful context. You'll also explore different ways of adding files, issues, and free-text notes to support precise, grounded interactions.

In this unit, you'll learn:

- How to create a Space and name it clearly for discoverability
- The difference between personal and organization-owned Spaces
- How to add and structure context through attachments and instructions
- Why organization and clarity improve Copilot's responses

## Creating a space

1. To create a space, go to https://github.com/copilot/spaces, and click Create space.
2. Give your space a name.
3. Choose whether the space is owned by you or by an organization you belong to. Organization-owned Spaces can be shared using GitHub's built-in permission model.
4. Optionally, add a description. This doesn't affect the responses Copilot gives in the space, but it can help others understand the context of the space.

   > **Note**
   >
   > You can change the name and description of your space at any time by clicking Edit in the top right corner of the space.

5. Click Save in the top right corner of the page.
6. Adding context to a space You can add two types of context to your space:

   Instructions: Free text that describes what Copilot should focus on within this space. Include its areas of expertise, what kinds of tasks it should help with, and what it should avoid. This helps Copilot give more relevant responses based on your intent.

**Attachments:** This context is used to provide more relevant answers to your questions. Additionally, Spaces always refer to the latest version of the code on the main branch of the repository.

To add attachments, click Add to the right of "Attachments", then choose one of the following options:

- Attach files and folders: You can add files and folders from your GitHub repositories. This includes code files, documentation, and other relevant content that can help Copilot understand the context of your space.
- Link pull requests and issues: You can paste the URLs of the GitHub issues and pull requests.
- Upload a file: You can upload files directly from your local machine. This includes images, text files, rich documents, and spreadsheets.
- Add text content: You can type or paste free-text content, such as transcripts, notes, or any other relevant information that can help Copilot understand the context of your space.

Congratulations! You just create a Space!

---

## Next unit: Sharing, Discoverability, and Governance

Previous Next

Ask Learn

100 XP

# Sharing, Discoverability, and Governance

5 minutes

To ensure your Space delivers lasting value, it must be easy to find, securely shared, and well-maintained. This unit focuses on how to manage visibility, follow GitHub permissions, and keep content fresh as your codebase evolves. You'll also learn lightweight governance strategies—from assigning ownership and adding usage guidance to setting up a review cadence—to ensure your Space remains accurate, discoverable, and aligned with team needs.

In this unit, you'll learn:

- How to manage visibility and sharing within your organization
- How GitHub permissions secure and govern access to linked content
- Why naming conventions and descriptions boost discoverability
- Best practices for Space ownership, freshness, and review cycles

## Visibility and sharing

Successful Spaces are easy to find, safe to share, and clearly "owned." When you create a Space, set visibility according to how broadly you intend others to use it. Depending on your environment, options may include keeping it under your personal ownership or making it visible to your organization.

Share the Space by link and, where available, rely on org-level browsing or catalogs to improve discoverability. Use a clear, purpose-driven title, and a short description that states the scope ("one job per Space"), intended audience, and expected outputs so teammates immediately know when to use it.

## Security and access

Security follows GitHub's existing permissions. A Space doesn't grant new access; it only surfaces content that viewers are already entitled to see. If a Space links to private repositories, issues, or pull requests, only users with the appropriate repo permissions see that material reflected in answers. This helps you share confidently across an organization while keeping sensitive information protected. As a best practice, avoid pasting sensitive data into free-text notes; prefer linking to version-controlled files where normal review and permissions apply.

## Versioning and freshness

Spaces stay fresh by referencing live GitHub sources. Linked files reflect the repository's default branch, and attached issues and pull requests evolve as they change, reducing the need to copy content into

separate documents. If you need branch-specific guidance or a historical snapshot, consider narrowing your references to the relevant files, adding a brief example in free text, or—if supported in your environment—attaching a text file that captures the exact content you want the Space to use. Keep the scope small so updates remain predictable and grounded.

# Governance

Treat governance as lightweight but intentional. Assign an owner who maintains the Space, add a short "How to use this Space" note at the top of the instructions, and include 1–3 canonical examples that define "good" output. Establish naming conventions (for example, "ServiceName—Onboarding Helper") and review cadence (for example, at each release) to prune stale sources and keep instructions aligned with reality. When a Space grows beyond a single job, split it into smaller Spaces so discoverability stays high and answer quality remains consistent.

Use this checklist when creating or updating a Space to keep it easy to find, safe to share, and reliably useful. Options (for example, org ownership, uploads) may vary by environment.

Naming and Purpose

- [ ] Choose a clear, purpose-driven title (for example, "ServiceName—Onboarding Helper"); keep "one job per Space."
- [ ] Write a 1–2 sentence description that states scope, intended audience, and expected outputs.
- [ ] Add a brief "How to use this Space" note at the top of the instructions.

Ownership and Visibility

- [ ] Set the correct owner (individual or organization, if available).
- [ ] Select appropriate visibility (private, org-visible, etc.).
- [ ] Verify access with a non-owner who expected GitHub permissions (Spaces inherit repo/issue/PR permissions).
- [ ] Share the URL and, where available, add collaborators.

Security and Privacy

- [ ] Don't paste sensitive data into free-text; prefer linking version-controlled files where normal review/permissions apply.
- [ ] Ensure all attached sources are suitable for the chosen visibility.
- [ ] If uploads are supported, limit the text content you’re comfortable sharing.
- [ ] Remove obsolete or confidential materials.

Discoverability and Docs

- [ ] Use consistent naming conventions across Spaces (team/service prefixes help).
- [ ] Add tags/keywords in the description to aid search.
- [ ] Announce or catalog the Space in your org’s preferred directory/channel.

Review Cadence and Governance

- [ ] Assign a maintainer/owner responsible for updates.
- [ ] Set a review cadence (for example, monthly or per release).
- [ ] At each review: validate links, test 2–3 representative prompts, update examples, prune noisy sources, and confirm visibility.
- [ ] Track feedback and improvement requests (issues, discussion, or a simple checklist in the description).

---

## Next unit: Do's and Don'ts of Working in a Space

Previous Next

Ask Learn

100 XP

# Do's and Don'ts of Working in a Space

5 minutes

Do keep your questions tightly scoped to the sources you attached—files, issues, pull requests, and notes—so answers stay grounded.

Don't @-mention people or other Copilot extensions in a Space: user mentions won't notify anyone, and extensions can't be invoked from Space chats.

Do treat the Space as a focused environment for a single task or domain, and reuse its own terminology to reinforce consistency.

Do use prompting patterns that lead to runnable, verifiable outputs. Start by confirming intent, then refine with concrete constraints (formats, time ranges, file paths, or sections to consider). Ask for executable code, queries, or commands and, when helpful, request references back to the included sources for traceability.

Don't expect the Space to pull in content that isn't included—unless your environment supports repository search you've explicitly attached, Copilot won't discover outside material.

Do iterate when responses drift: tighten instructions, add one to three high-quality examples that demonstrate "good" output, and prune noisy or irrelevant sources.

Don't let the Space sprawl beyond a single job or exceed model context limits; if you hit size warnings or degraded answers, reduce sources or split into smaller Spaces to restore precision and predictability.

Do keep context fresh and well-ordered. Link version-controlled files so the Space reflects your repository's default branch as it evolves, and lead with the most important sources or examples since ordering can influence responses.

Don't paste sensitive data into free-text notes; prefer linking to files in repos (or use uploads where supported) so standard review and permissions apply.

## Next unit: Exercise - Democratize tribal knowledge using Copilot Spaces

Ask Learn

100 XP

# Exercise - Democratize tribal knowledge using Copilot Spaces

5 minutes

In this exercise, you will:

- Add a repository as a source to your Copilot Space
- Explore and summarize project management process documentation
- Create and use custom chat modes for deep process analysis
- Update repository documentation based on insights discovered

## Getting started

When you select the Start the exercise on GitHub button below, you'll be navigated to a public GitHub template repository that prompts you to complete a series of small challenges. When you've finished the exercise in GitHub, return here for: A quick knowledge check. A summary of what you've learned. A badge for completing this module.

Start the exercise on GitHub

## Next unit: Module assessment

Ask Learn

200 XP

# Module assessment

4 minutes

**1. When is a GitHub Copilot Space the better choice than general or repo-wide chat?**

- When you need broad discovery across many repositories
- When you want consistent, reproducible answers on a tightly scoped topic
- When you want Copilot to automatically discover any relevant content in your org
- When you have no specific task or domain in mind

**2. Which statement about Space ownership and description is true?**

- A Space must be organization-owned; personal Spaces aren't supported
- The description changes Copilot's answers in the Space
- You can choose personal or organization ownership (where available), and the description is for human readers
- Ownership can't be changed once set, and descriptions affect answer quality

**3. Which action does NOT add usable context for Copilot in a Space?**

- Attaching files or folders from a GitHub repository
- Pasting URLs of GitHub issues and pull requests
- Uploading a local file (for example, a text document or spreadsheet)
- @-mentioning a Copilot extension so it can run in Space chat

**4. How do Spaces handle security and access to linked items?**

- A Space grants temporary read access to all linked private repositories
- A Space mirrors GitHub permissions and surfaces only what a viewer can already see
- A Space creates a copy of private content that anyone with the link can access

- A Space requires repo admins to approved list each viewer manually

**5. You need branch-specific guidance or a historical snapshot in a Space. What should you do?**

- Change the repository's default branch to lock Space answers to that branch
- Rely on general chat to retrieve older versions automatically
- Narrow references to relevant files and add a brief example, or attach a text file with the exact content
- Paste the sensitive historical content into free-text notes for convenience

**6. Which prompting pattern best supports runnable, verifiable outputs in a Space?**

- Ask for a summary without constraints to keep the model creative
- Confirm intent, add concrete constraints (formats, ranges, file paths), and request executable outputs with references
- Use many broad instructions to widen the context as much as possible
- Avoid referencing attached sources to prevent overfitting

**7. You notice size warnings and increasingly vague answers from your Space. What's the best next step?**

- Add more examples to increase context so the model has more to learn from
- Reduce sources or split the Space into smaller, single-job Spaces
- Start @-mentioning people to pull in their expertise
- Turn off repository linking so the Space doesn't change

Submit answers

## Next unit: Summary

Previous Next